

지갑 프로비저닝 + 매핑 설계

userId ↔ walletAddr 연결의 설계 원칙
VASP 분기 · 먹등성 · Repository 패턴

WalletProvisioningService

DIP

PostgreSQL

COINCRAFT
ACADEMY

전체 아키텍처 맵

ARCHITECTURE REMINDER — 신뢰 경계로 시스템을 나눈다 · M5는 레이어 1 진입점 · 레이어 3 비즈니스 로직

내부망

레이어 1 — 사용자 **M5 진입점**

Kyobo App

↓ 사용자 이벤트 / API 응답

레이어 2 — 게이트웨이

Webhook
Receiver

Redis Event
Pipeline

IVASP
Adapter

IBlockchain
Adapter

↓ 발행 요청 / 원장 업데이트

레이어 3 — 코어 서비스

비즈니스 로직

데이터 관리

거버넌스

인터넷

레이어 5 — 퍼블릭 블록
체인

이더리움 EVM

↓ Tx Broadcast / 이벤트 감지

레이어 4 — 인가 VASP

인가 VASP

↑ 발행 요청 / Webhook 이벤트 ↓

M5 핵심 구간: 레이어 1 사용자 진입점(POST /api/wallet/provision) · 레이어 3 비즈니스 로직 — WalletProvisioningService / VASP 분기(EXTERNAL-KYOBO) /
역등성 / Repository 패턴 / DIP

두 개의 식별자 공간

두 세계 · 지갑 프로비저닝 정의

교보 내부 세계

```
userId = "K-20240001"  
// SSO 기반 고객 ID  
// 사람이 읽을 수 있음  
// 고객임을 식별 가능
```

교보생명 내부 인증 시스템이 부여하는 식별자. 블록체인에서는 아무 의미 없음.

블록체인 세계

```
walletAddr =  
  "0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266"  
// ECDSA 공개키의 Keccak-256 해시  
// NFT는 이 주소로만 발행 가능
```

익명성. 주소만 보고는 누구인지 알 수 없음.

K-20240001

provision()

0xf39Fd6e5...

지갑 프로비저닝 = 두 식별자를 최초 1회 연결해서 DB에 저장하는 작업.
이후 NFT 발행 시 DB만 조회한다. VASP를 매번 호출하지 않는다.

userId를 블록체인에 직접 저장하면 안 되는 이유

"매핑 테이블 없이 컨트랙트에 저장하면 되지 않나?" — 안 된다

이유	설명
개인정보보호법	"K-20240001"은 교보 고객임을 식별 가능. 공개 블록체인 = 삭제 불가 — GDPR · 개인정보보호법 위반
지갑 교체 불가	지갑을 잃어버리면 컨트랙트를 수정해야 함. 오프체인 테이블은 UPDATE 가능
체인 전환 유연성	EVM → XRPL로 바뀌면 주소 형식 자체가 달라짐. 매핑 테이블만 업데이트 하면 비즈니스 로직은 그대로

❌ 온체인 저장 시

- 블록체인은 불변 — 삭제 불가
- "K-20240001" 노출 = 고객 신원 노출
- 지갑 변경 시 컨트랙트 업그레이드
- 체인 전환 시 전체 재설계

✅ 오프체인 매핑 테이블

- DB는 삭제·수정 가능 → 규정 준수
- 블록체인에는 walletAddr만 공개
- 지갑 교체 = UPDATE 한 줄
- 체인 전환 = 매핑 테이블만 교체

앱이 먼저, 서버가 나중

provision()의 역할 — "생성"이 아니라 "조회"

- ① **사용자가 앱 실행** — VASP 지갑 SDK 초기화 → 사용자 고유 지갑 주소 생성/로드
이 시점에서 지갑은 이미 존재한다
- ② **앱 → 서버 지갑 등록 요청**

```
POST /api/wallet/provision { userId, vaspType: 'EXTERNAL' }
```
- ③ **서버 → VASP API 이미 만들어진 주소를 가져옴**

```
vaspClient.getWalletAddr(userId) ← "생성"이 아니라 "조회"
```
- ④ **서버 DB에 저장** — userId ↔ walletAddr 매핑 완료
이후 NFT 발행 시 DB만 조회. VASP 재호출 없음.

EXTERNAL (Phase 1)

`getWalletAddr()` — 앱 SDK가 이미 만든 주소를 VASP에서 조회

KYOBO (Phase 4)

`createWallet()` — 교보 HSM이 직접 키 생성. 앱 SDK 없음.

user_wallet_mapping 테이블 설계 1/2

DB 스키마

```
CREATE TABLE user_wallet_mapping (  
  id          BIGSERIAL PRIMARY KEY,  
  user_id     VARCHAR(64) NOT NULL UNIQUE, -- UNIQUE: userId당 1개 지갑만 허용 (Phase 1)  
  wallet_addr VARCHAR(42) NOT NULL,        -- Ethereum 주소: 0x + 40자  
  vasp_type   VARCHAR(16) NOT NULL,        -- 'EXTERNAL' | 'KYOBO'  
  verified    BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  CONSTRAINT chk_vasp_type CHECK (vasp_type IN ('EXTERNAL', 'KYOBO'))  
);  
  
-- 지갑 주소로 역방향 조회 인덱스 (누가 이 지갑을 소유하는지)  
CREATE INDEX idx_wallet_mapping_addr  
  ON user_wallet_mapping(wallet_addr);
```

UNIQUE(user_id) — DB 레벨에서 1:1 강제. 코드 레벨만으로는 부족하다 — 동시 요청 두 개가 동시에 통과할 수 있다 (TOCTOU).

user_wallet_mapping 테이블 설계 2/2

설계 결정 — 1:1 구조 (Phase 1) · verified 컬럼

1:1 vs 1:N 구조 결정

구조	장점	단점	결정
1:1 userId → 1개 주소	단순, 발행 대상 명확	다중 지갑 불가	Phase 1 채택
1:N userId → 여러 주소	디바이스별 지갑 분리	발행 대상 주소 선택 로직 복잡	Phase 2+

verified 컬럼이 존재하는 이유

Phase 1 (현재) — DEFAULT TRUE

수탁(Custodial) 방식만 지원.

VASP 또는 교보가 지갑을 직접 생성

→ 소유권이 자명 → **verified = true** 자동

Phase 2+ — 자가관리 지갑

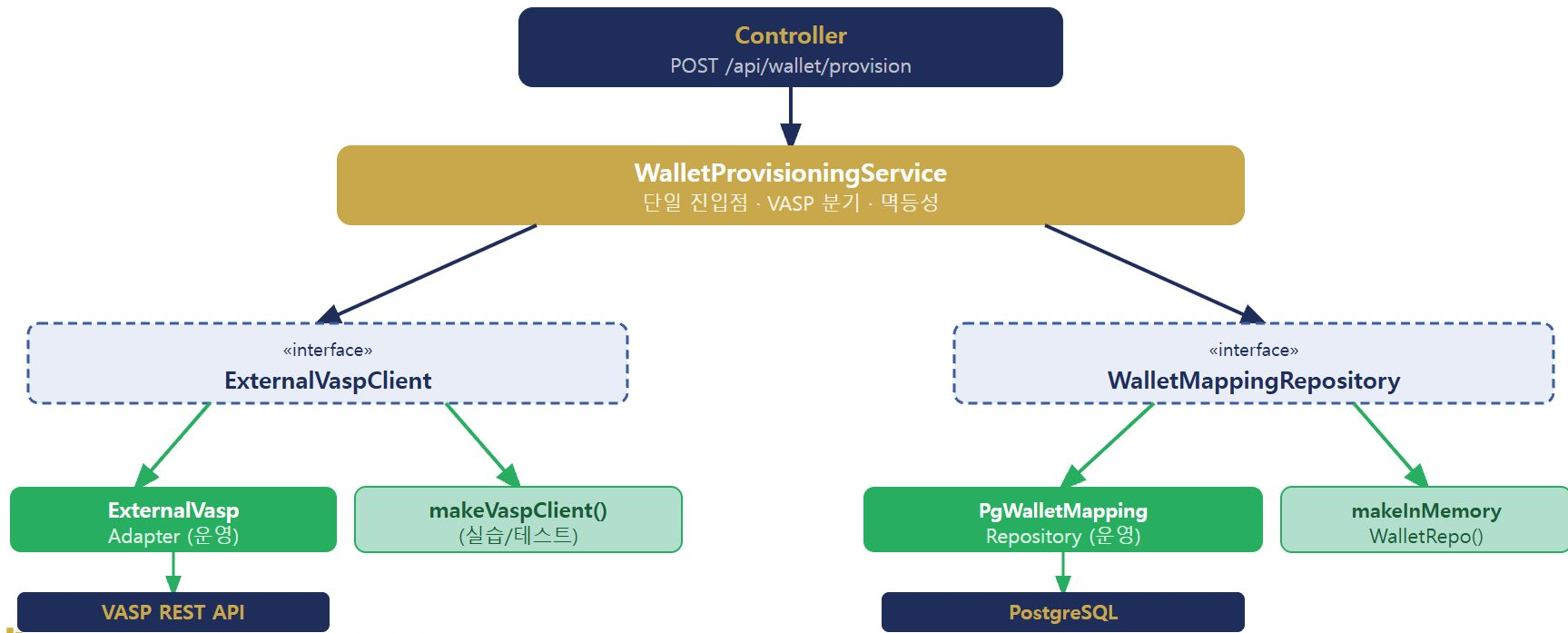
사용자가 MetaMask 등 외부 지갑 연결.

소유권 검증(서명 확인) 전까지 **verified = false**.

컬럼이 미리 있어야 ALTER TABLE 없이 바로 사용 가능.

전체 의존 관계 1/2

계층 구조 순서도



전체 의존 관계 2/2

WalletProvisioningService의 두 가지 의존 관계 · DIP

두 가지 의존 관계

1 ExternalVaspClient

VASP API 호출 — 지갑 주소 획득

2 WalletMappingService

획득한 주소를 DB에 저장·조회

WalletProvisioningService는 직접 SQL을 쓰거나
VASP HTTP 클라이언트를 생성하지 않는다.

auditLog는 선택적(optional). `auditLog?.record()`
주입 안 해도 동작 — Optional Chaining

DIP — 의존성 역전 원칙

M4와 동일한 원칙

M4: `AuditLogService` → `DatabaseClient`

M5: `WalletMappingService` → `WalletMappingRepository`

고수준 모듈이 저수준 모듈에 직접 의존하지 않는다.
둘 다 추상화(인터페이스)에 의존한다.

효과

- PostgreSQL → DynamoDB 교체 시 Service 코드 무변경
- 테스트: InMemory 구현체로 실제 DB 없이 검증
- Phase 전환: VASP Adapter만 교체

WalletMappingService 1/4

Repository 패턴 · interface WalletMappingRepository

서비스 코드에 SQL이 없다.
M4 AuditLogService 와 동일한 원칙.

```
interface WalletMappingRepository {  
  
    findByUserId(  
        userId: string  
    ): Promise<WalletMapping | null>;  
  
    save(  
        mapping: WalletMapping  
    ): Promise<void>;  
  
    findByWalletAddr(  
        walletAddr: string  
    ): Promise<WalletMapping | null>;  
}
```

WalletMapping 모델

```
interface WalletMapping {  
    userId: string;  
    walletAddr: string;  
    vaspType: VaspType;  
    verified: boolean;  
    createdAt: Date;  
}
```

3개 메서드의 역할

메서드	호출 시점
findByUserId	역등성 체크, 발행 전 주소 조회
save	프로비저닝 완료 후 저장
findByWalletAddr	역방향 조회 (감사, 분쟁)

WalletMappingService 2/4

getWalletAddr — NFT 발행 직전 주소 확인

```
// NFT 발행 직전 - 발행 대상 주소 확인
async getWalletAddr(
  userId: string
): Promise<string> {

  const existing =
    await this.repo.findByUserId(userId);

  if (!existing)
    throw new WalletNotFoundError(userId);

  if (!existing.verified)
    throw new WalletNotVerifiedError(userId);

  return existing.walletAddr;
}
```

두 가지 에러 경로

WalletNotFoundError → HTTP 404

DB에 매핑이 없음.

프로비저닝이 완료되지 않은 사용자.

WalletNotVerifiedError → HTTP 403

매핑은 있지만 verified = false.

Phase 2+ 자가관리 지갑 미검증 상태.

자동 프로비저닝을 하지 않는 이유

지갑 생성은 명시적 provision()을 통해야만 함.

운영자 모르게 새 지갑이 생기면 감사 추적 불가.

WalletMappingService 3/4

getMapping — 프로비저닝 중복 체크

```
// @ 프로비저닝 중복 체크
async getMapping(
  userId: string
): Promise<WalletMapping | null> {

  return this.repo.findById(userId);
}
```

단순한 위임(delegation).
provision() 가장 첫 번째 단계에서 호출.

멥등성 구현의 핵심

- 1 getMapping() 호출 — DB SELECT
- ↓ null → VASP 호출 → DB 저장 진행
- ↓ WalletMapping 반환 → 즉시 반환
VASP 호출 없음. INSERT 없음.

같은 userId로 provision()을 10번 호출해도
VASP는 1번만, DB는 1행만.

WalletMappingService 4/4

saveMapping — 매핑 저장

```
// © 매핑 저장 (WalletProvisioningService가 호출)
```

```
async saveMapping(  
  userId:    string,  
  walletAddr: string,  
  vaspType:  VaspType,  
): Promise<void> {  
  
  await this.repo.save({  
    userId,  
    walletAddr,  
    vaspType,  
    verified: true,  
    createdAt: new Date(),  
  });  
}
```

호출 순서의 중요성

- ① VASP 호출 → walletAddr 획득
- ② **saveMapping() 호출**
VASP 성공 후에만 실행
- ③ 감사 로그 기록

VASP 실패 시 saveMapping()이 호출되지 않는다.
DB에 부분 저장 없음 → 일관 상태 유지.

verified: true — Phase 1은 항상 수탁 방식이므로 검증 불필요.

PgWalletMappingRepository 1/5

SQL 구현체 — 클래스 선언부 · 생성자

```
export class PgWalletMappingRepository
  implements WalletMappingRepository {

  constructor(
    private readonly pool: Pool
  ) {}

  // findById()      → 슬라이드 15
  // save()          → 슬라이드 16
  // findByWalletAddr() → 슬라이드 17
  // _toModel()      → 슬라이드 18
}
```

implements 키워드

implements WalletMappingRepository

TypeScript 컴파일러가 인터페이스의 모든 메서드가 구현되었는지 컴파일 타임에 검사. 메서드 누락 시 즉시 에러.

Pool 생성자 주입

의존성 주입

Pool을 직접 생성하지 않고 주입받는다.

- 테스트 시 가짜 Pool 주입 가능
- 연결 풀 설정을 외부에서 제어 가능
- DIP 원칙과 일관됨

PgWalletMappingRepository 2/5

findByUserId

```
async findByUserId(  
  userId: string  
): Promise<WalletMapping | null> {  
  
  const { rows } = await this.pool.query(  
    'SELECT * FROM user_wallet_mapping'  
    + ' WHERE user_id = $1',  
    [userId],  
  );  
  
  return rows[0]  
    ? this._toModel(rows[0])  
    : null;  
}
```

포인트

- 1 **Parameterized Query** — `$1` 플레이스홀더
SQL Injection 방지. 값은 별도 배열로 전달.
- 2 **rows[0] 존재 여부로 null 처리**
없으면 null 반환 — 에러를 throw하지 않음.
에러 판단은 호출자(getWalletAddr)가 한다.
- 3 **_toModel() 변환**
DB row (snake_case) → 도메인 모델 (camelCase)

PgWalletMappingRepository 3/5

save — UPSERT

```
async save(  
  mapping: WalletMapping  
) : Promise<void> {  
  await this.pool.query(  
    `INSERT INTO user_wallet_mapping  
      (user_id, wallet_addr, vasp_type, verified, created_at)  
      VALUES ($1, $2, $3, $4, $5)  
      ON CONFLICT (user_id)  
      DO UPDATE SET  
        wallet_addr = $2,  
        vasp_type    = $3,  
        verified     = $4`,  
    [  
      mapping.userId,  
      mapping.walletAddr,  
      mapping.vaspType,  
      mapping.verified,  
      mapping.createdAt,  
    ],  
  );  
}
```

ON CONFLICT DO UPDATE — UPSERT

- 1 INSERT 먼저 시도
- 2 user_id 충돌 발생 → DO UPDATE
지갑 교체 시나리오 처리 (Phase 2+)

SELECT + UPDATE 없이 단일 왕복.

TOCTOU 문제 없음 — DB가 원자적으로 처리.

Phase 1에서는 UNIQUE(user_id)로 두 번째 INSERT가 오지 않지만, 구현체는 교체 시나리오도 안전하게 처리.

PgWalletMappingRepository 4/5

findByWalletAddr — 역방향 조회

```
async findByWalletAddr(
  walletAddr: string
): Promise<WalletMapping | null> {

  const { rows } = await this.pool.query(
    'SELECT * FROM user_wallet_mapping'
    + ' WHERE wallet_addr = $1',
    [walletAddr.toLowerCase()], // lowercase 정규화
  );

  return rows[0]
    ? this._toModel(rows[0])
    : null;
}
```

역방향 조회의 용도

언제 필요한가?

- 블록체인 TX에서 발신 주소를 확인했을 때
- 감사 로그에서 walletAddr로 userId를 역추적
- 분쟁 발생 시 특정 주소 소유자 확인

toLowerCase() 정규화

Ethereum 주소는 대소문자가 혼재(EIP-55 체크섬). DB에는 소문자로 저장하고 조회 시도 소문자로 정규화해서 일관성 보장.

인덱스: idx_wallet_mapping_addr ON
user_wallet_mapping(wallet_addr)
전체 테이블 스캔 없이 빠른 역조회 가능.

PgWalletMappingRepository 5/5

_toModel — DB row → 도메인 모델 변환

```
private _toModel(  
    row: Record<string, unknown>  
) : WalletMapping {  
  
    return {  
        userId:  
            row['user_id'] as string,  
        walletAddr:  
            row['wallet_addr'] as string,  
        vaspType:  
            row['vasp_type'] as VaspType,  
        verified:  
            row['verified'] as boolean,  
        createdAt:  
            new Date(row['created_at'] as string),  
    };  
}
```

왜 _toModel이 필요한가?

1 snake_case → camelCase

user_id → userId, wallet_addr → walletAddr

2 타입 변환

created_at (string) → Date 객체

3 도메인 모델 분리

DB 스키마 변경이 도메인 인터페이스에 영향 안 줌

private — 리포지토리 내부 구현 세부사항.
외부에서 직접 호출 불가. 인터페이스에 노출 안 됨.

WalletProvisioningService 1/4

VASP 분기와 단일 진입점 — 나쁜 설계 vs 좋은 설계

✗ 나쁜 설계

```
// VASP가 바뀔 때마다 컨트롤러 수정 필요
app.post('/api/wallet/provision',
  async (req, res) => {

    if (req.body.vaspType === 'EXTERNAL') {
      // VASP A 전용 로직...
    } else if (req.body.vaspType === 'KYOBO') {
      // VASP B 전용 로직...
    }

    // VASP 추가할 때마다 여기를 수정
  });
```

✓ 좋은 설계

```
// 컨트롤러는 단순 위임
app.post('/api/wallet/provision',
  async (req, res) => {

    const { userId, vaspType } = req.body;
    const result = await
      walletProvisioningService
        .provision(userId, vaspType);

    res.status(200).json(result);
  });
```

VASP 분기 로직은 Service 내부로 숨긴다.
컨트롤러는 라우팅만. 비즈니스 로직 없음.

WalletProvisioningService 2/4

provision() 내부 분기 — 생성자

```
export class WalletProvisioningService {  
  
  constructor(  
    private readonly vaspClient:  
      ExternalVaspClient,  
    private readonly walletMapping:  
      WalletMappingService,  
    private readonly auditLog?:  
      AuditLogAdapter,  
  ) {}  
  
  async provision(  
    userId: string,  
    vaspType: VaspType | string,  
  ): Promise<ProvisionResult> { ... }  
}
```

생성자 3개 파라미터

- 1 **vaspClient** (필수)
ExternalVaspClient 인터페이스 — VASP 호출
- 2 **walletMapping** (필수)
WalletMappingService — DB 저장·조회
- 3 **auditLog?** (선택적)
AuditLogAdapter — 감사 로그, 주입 안 해도 동작.

VaspType | string — string도 허용해서
미지원 타입 입력 시 UnsupportedVaspError를 throw할 수
있게 함.

WalletProvisioningService 3/4

provision() — ① 멍등성 · ② VASP 분기

```
async provision(  
  userId: string,  
  vaspType: VaspType | string,  
) : Promise<ProvisionResult> {  
  
  // ① 멍등성 - DB에 이미 있으면 즉시 반환  
  const existing =  
    await this.walletMapping.GetMapping(userId);  
  
  if (existing) {  
    return {  
      userId,  
      walletAddress: existing.walletAddr,  
      vaspType: existing.vaspType,  
      provisionedAt: existing.createdAt,  
    };  
  }  
  
  // ② VASP 분기  
  let walletAddress: string;  
  if (vaspType === 'EXTERNAL')  
    walletAddress = await this.vaspClient.getWalletAddr(userId);  
  else if (vaspType === 'KYOBO')  
    walletAddress = await this.vaspClient.createWallet(userId);  
  else  
    throw new UnsupportedVaspError(vaspType);  
}
```

① 멍등성 포인트

가장 먼저 DB를 조회한다.
있으면 VASP를 호출하지 않고 즉시 반환.
앱 재시작·네트워크 재시도에 안전.

② VASP 분기 포인트

EXTERNAL → `getWalletAddr()` 조회

KYOBO → `createWallet()` 생성

그 외 → `UnsupportedVaspError` throw

미지원 VASP 타입은 **DB 저장 전에** 에러 발생.
→ DB에 잘못된 데이터 없음.

WalletProvisioningService 4/4

provision() — ③ DB 저장 · ④ 감사 로그

```
// ③ DB 저장 - VASP 호출 성공 후에만
await this.walletMapping.saveMapping(
  userId, walletAddress, vaspType as VaspType
);

// ④ 감사 로그 (선택적, Optional Chaining)
await this.auditLog?.record({
  actor:      'system',
  action:     'WALLET_PROVISIONED',
  resourceType: 'USER',
  resourceId:  userId,
  afterState:  { walletAddress, vaspType },
});

return {
  userId, walletAddress,
  vaspType:    vaspType as VaspType,
  provisionedAt: new Date(),
};
}
```

③ DB 저장 순서의 의미

VASP 실패 시 여기까지 오지 않는다.
→ DB 미저장 = 일관 상태 유지.

④ 감사 로그 포인트

`auditLog?.record()` — Optional Chaining.
auditLog가 undefined여도 에러 없이 스킵.
테스트에서 주입 생략 가능.

멥등성 early return 시 감사 로그 **미기록**.
재요청은 새 이벤트가 아님 — 중복 기록 방지.

ExternalVaspClient 인터페이스 1/2

Phase 전환 설계 — interface ExternalVaspClient

```
interface ExternalVaspClient {  
  
  // Custodial 지갑 주소 조회 (Phase 1)  
  getWalletAddr(  
    userId: string  
  ): Promise<string>;  
  
  // 신규 지갑 생성 (Phase 4 KYOBO)  
  createWallet(  
    userId: string  
  ): Promise<string>;  
  
}
```

두 메서드 모두 `userId: string → Promise<string>` .
반환값은 항상 `walletAddr`.

실습에서의 구현

```
// EXTERNAL - BIP-44 HD 파생 (결정론적)  
const path = `m/44'/60'/0'/0/${hdIdx++}`;  
const child = hdRoot.derivePath(path);  
// 같은 니모닉 + 같은 인덱스 = 항상 같은 주소  
  
// KYOBO - HSM 신규 생성 (비결정론적)  
const wallet = ethers.Wallet.createRandom();  
// wallet.privateKey → HSM 보관, 외부 노출 금지
```

HD 파생: 같은 니모닉 + 인덱스 → 같은 주소.
먹등성의 근거 — 재시도해도 같은 주소.

ExternalVaspClient 인터페이스 2/2

Phase 전환 설계 — Phase1 vs Phase4 비교 · 어댑터 사용 예시

Phase	구현체	동작	키 보관
Phase 1 (현재)	ExternalVaspAdapter	VASP REST API 호출 — 앱이 만든 주소 조회	VASP
Phase 4	KyoboCustodyAdapter	교보 자체 HSM/MPC — 직접 키 생성	교보 HSM

Phase 전환 시 코드 변경 범위

```
// Phase 1
new WalletProvisioningService(
    new ExternalVaspAdapter(apiKey),
    walletMapping,
);
```

```
// Phase 4 (이 한 줄만 바뀜)
new WalletProvisioningService(
    new KyoboCustodyAdapter(hsm),
    walletMapping,
);
```

WalletProvisioningService 코드는 한 글자도 바뀌지 않는다.
생성자에 주입하는 구현체만 교체. DIP의 핵심 효과.

에러 타입 설계 — 계층별 책임

instanceof 판별 → HTTP 상태코드 매핑

에러 클래스	발생 시점	HTTP	이유
UnsupportedVaspError	미지원 vaspType 입력	400	클라이언트 잘못된 요청
WalletNotFoundError	getWalletAddr() — DB에 매핑 없음	404	리소스 없음
WalletNotVerifiedError	getWalletAddr() — verified=false	403	소유권 미검증
Error('VASP network timeout')	vaspClient 호출 실패	502	업스트림 서버 오류

에러 클래스를 별도로 선언하는 이유: 컨트롤러가 instanceof 로 타입을 판별해서 적절한 HTTP 상태코드를 반환하기 위해서다.

```
export class UnsupportedVaspError extends Error {
  constructor(vaspType: string) {
    super(`지원하지 않는 VASP 타입: ${vaspType}`);
    this.name = 'UnsupportedVaspError';
  }
}
```

HTTP API 설계 — 컨트롤러 1/2

POST /api/wallet/provision — 최초 등록

```
app.post(
  '/api/wallet/provision',
  async (req, res) => {
    try {
      const { userId, vaspType } = req.body;
      const result = await
        walletProvisioningService
          .provision(userId, vaspType);

      res.status(200).json(result);

    } catch (err) {
      if (err instanceof UnsupportedVaspError)
        return res.status(400)
          .json({ error: err.message });

      if (err instanceof WalletNotFoundError)
        return res.status(404)
          .json({ error: err.message });

      res.status(500)
        .json({ error: 'Internal server error' });
    }
  }
);
```

에러 처리 계층

- 1 **UnsupportedVaspError → 400**
클라이언트가 잘못된 vaspType 전송
- 2 **WalletNotFoundError → 404**
매핑이 없는 userId 조회
- 3 **나머지 → 500**
예측 불가 에러. 내부 정보 노출 안 함.

幂등성 보장 — 이미 프로비저닝된 userId는 200을 반환하며 기존 주소를 그대로 돌려준다.

HTTP API 설계 — 컨트롤러 2/2

GET /api/wallet/:userId — 현재 등록 주소 조회

```
app.get(
  '/api/wallet/:userId',
  async (req, res) => {
    try {
      const addr = await
        walletMappingService
          .getWalletAddr(req.params.userId);

      res.status(200).json({ walletAddr: addr });

    } catch (err) {
      if (err instanceof WalletNotFoundError)
        return res.status(404)
          .json({ error: err.message });

      if (err instanceof WalletNotVerifiedError)
        return res.status(403)
          .json({ error: err.message });

      res.status(500)
        .json({ error: 'Internal server error' });
    }
  }
);
```

두 API의 역할 분리

엔드포인트	목적
POST /provision	최초 1회 — VASP 호출 + DB 저장
GET /:userId	조회만 — DB에서만 읽음

GET의 에러 경로

WalletNotFoundError → 404

프로비저닝이 안 된 사용자.

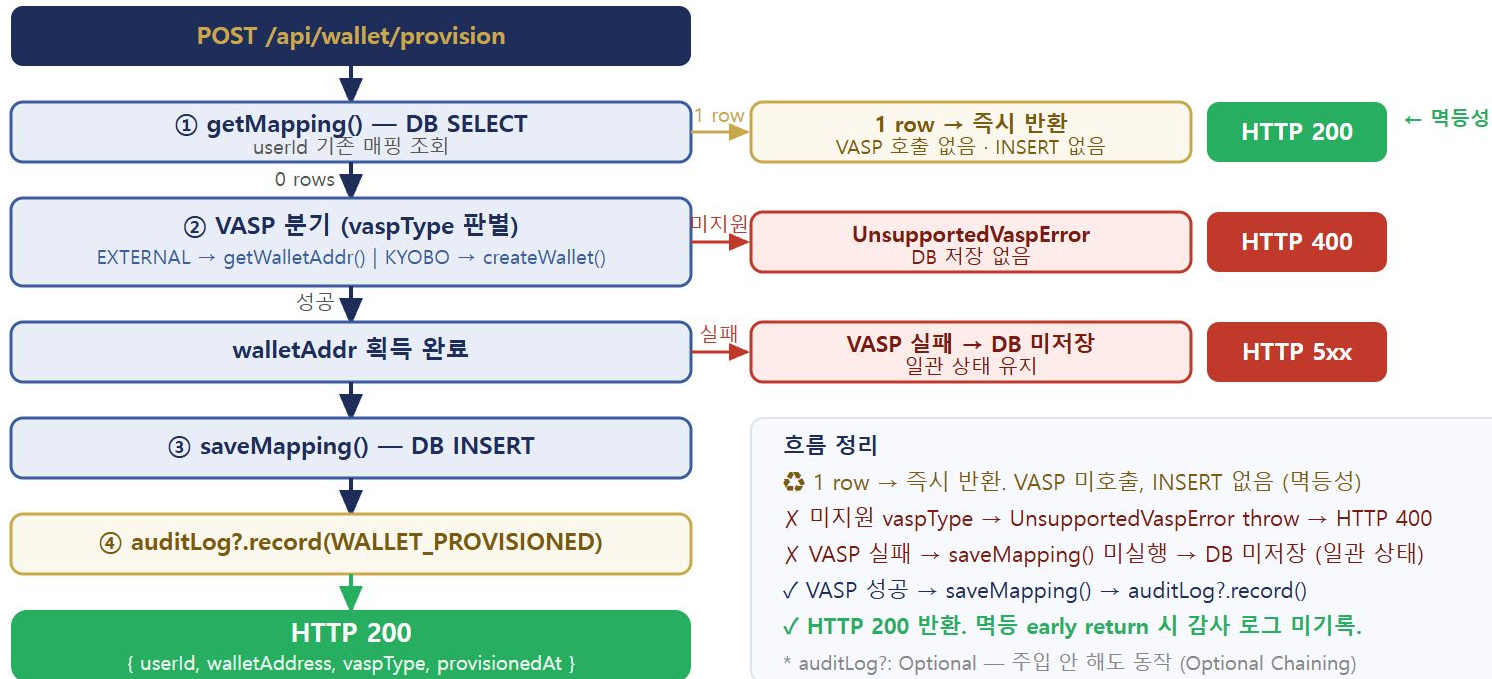
WalletNotVerifiedError → 403

매핑은 있지만 소유권 미검증 상태.

GET은 절대 VASP를 호출하지 않는다.
조회는 항상 DB만 사용.

provision() 완전한 흐름 정리

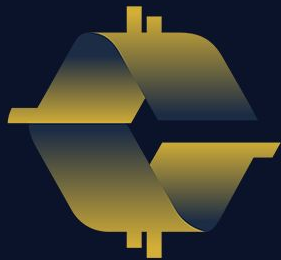
POST /api/wallet/provision { userId: 'K-20240001', vaspType: 'EXTERNAL' }



실습

S27 실습 — 7개 섹션

- [1] EXTERNAL HD 지갑 파생
- [2] KYOBO HSM 신규 생성
- [3] 미지원 vaspType → UnsupportedVaspError
- [4] 컨트롤러 계층 시뮬레이션
- [5] 맥등성 — 동일 userId 3회 요청
- [6] VASP 실패 → DB 미저장
- [7] 감사 로그 + SHA-256 체크섬



COINCRAFT
ACADEMY

[1] EXTERNAL — HD 지갑 파생

`npm run exercise:s27:1 · vaspType = 'EXTERNAL' → getWalletAddr(userId)`

```
$ npm run exercise:s27:1
```

실행 코드

```
const result = await svc.provision('K-20240001', 'EXTERNAL');  
// 내부 흐름  
// 1. walletMapping.getMmapping() → null (최초)  
// 2. vaspClient.getWalletAddr() → HD 파생 경로 계산  
//    path = m/44'/60'/0'/0/0  
// 3. walletMapping.saveMapping() → INSERT
```

예상 출력

```
[VASP] HD 파생 path=m/44'/60'/0'/0/0  
[VASP] address 0xABC...  
[DB] INSERT INTO user_wallet_mapping → 1 row  
result.walletAddress : 0xABC...  
vasp calls : 1
```

핵심 포인트

HD 파생 경로: BIP-44 `m/44'/60'/0'/0/{idx}`
동일 니모닉에서 `userId`별 고유 주소 결정론적 생성

getWalletAddr vs createWallet:

EXTERNAL → 기존 custody 주소 조회(또는 파생)
KYOBO → 신규 HSM 키 생성

DB 확인:

```
SELECT * FROM user_wallet_mapping WHERE user_id = 'K-20240001'  
vasp_type = 'EXTERNAL', verified = true
```

[2] KYOBO — HSM 신규 지갑 생성

npm run exercise:s27:2 · vaspType = 'KYOBO' → createWallet(userId)

```
$ npm run exercise:s27:2
```

실행 코드

```
const result = await svc.provision('K-20240002', 'KYOBO');  
// 내부 흐름  
// 1. walletMapping.getMmapping() → null  
// 2. vaspClient.createWallet() → 랜덤 키페어 생성  
//    privateKey → HSM 보관, address만 반환  
// 3. walletMapping.saveMapping() → INSERT
```

예상 출력

```
[KYOBO HSM] userId      K-20240002  
[KYOBO HSM] address     0xDEF...  
[KYOBO HSM] privateKey  0x... ← HSM 보관, 외부 노출 금지  
[DB]   INSERT INTO user_wallet_mapping → 1 row  
vasp_type = 'KYOBO'
```

EXTERNAL vs KYOBO 비교

항목	EXTERNAL	KYOBO
호출 메서드	getWalletAddr	createWallet
키 관리	외부 VASP custody	교보 HSM
주소 생성	결정론적 HD 파생	랜덤 생성
재현 가능	O (같은 경로)	X (1회성)

설계 원칙: privateKey는 절대 서비스 레이어로 올라오지 않는다.
HSM이 서명까지 처리.

[3] 미지원 vaspType → UnsupportedVaspError

npm run exercise:s27:3 · 알 수 없는 VASP → 타입 에러 분기

```
$ npm run exercise:s27:3
```

실행 코드

```
try {
  await svc.provision('K-20240003', 'LEGACY_VASP');
} catch (err) {
  console.log(err.constructor.name); // UnsupportedVaspError
  console.log(err.message);          // 지원하지 않는 VASP 타입: LEGACY_VASP
}
console.log('saveCalls:', mapping.saveCalls); // 0
console.log('DB rows:',    check.rows[0].count); // 0
```

에러 클래스

```
class UnsupportedVaspError extends Error {
  constructor(vaspType: string) {
    super(`지원하지 않는 VASP 타입: ${vaspType}`);
    this.name = 'UnsupportedVaspError';
  }
}
```

핵심 포인트

에러 발생 순서:

- ① getMapping() → null 확인
- ② if/else if/else 분기 → else 도달
- ③ throw UnsupportedVaspError
- ④ saveMapping() 호출 안 됨 → DB 미저장

컨트롤러 매핑 규칙:

UnsupportedVaspError → HTTP 400

기타 에러 → HTTP 500

일관 상태 보장: VASP 에러 시 DB INSERT 없음 → 원
작성 유지

[4] 컨트롤러 계층 — 에러 → HTTP 상태코드

npm run exercise:s27:4 · UnsupportedVaspError → 400 / 나머지 → 200 or 500

```
$ npm run exercise:s27:4
```

simulateController

```
async function simulateController(  
  userId: string, vaspType: string  
): Promise<number> {  
  try {  
    await svc.provision(userId, vaspType);  
    return 200;  
  } catch (err) {  
    if (err instanceof UnsupportedVaspError)  
      return 400;  
    return 500;  
  }  
}
```

예상 출력

```
EXTERNAL    : 200  
LEGACY_VASP : 400  
KYOBO       : 200
```

에러 계층 매핑

vaspType	결과	HTTP
EXTERNAL	정상 프로비저닝	200
KYOBO	정상 프로비저닝	200
LEGACY_VASP	UnsupportedVaspError	400
기타 에러	서버 장애	500

설계 원칙: 서비스 레이어는 도메인 에러를 던진다. 컨트롤러만 HTTP 상태코드를 안다. 계층 간 관심사 분리.

[5] 멍등성 — 동일 userId 3회 요청

npm run exercise:s27:5 · 같은 userId로 몇 번 호출해도 결과·DB 동일

```
$ npm run exercise:s27:5
```

실행 코드

```
const r1 = await svc.provision('K-20240007', 'EXTERNAL');  
const r2 = await svc.provision('K-20240007', 'EXTERNAL');  
const r3 = await svc.provision('K-20240007', 'EXTERNAL');
```

예상 출력

```
r1.walletAddress : 0xABC...  
r2.walletAddress : 0xABC... ← 동일  
r3.walletAddress : 0xABC... ← 동일  
vasp calls : 1 ← 최초 1회만  
saveCalls : 1 ← INSERT 1회만  
DB rows : 1 ← 3회 요청에도 1행
```

멍등성 구현 흐름

1회차: getMapping() → null → VASP 호출 → saveMapping() → INSERT

2·3회차: getMapping() → 기존 row 반환 → 즉시 리턴
VASP 호출 없음 / INSERT 없음

멍등성의 의미: 네트워크 재시도·중복 요청이 와도 데이터 정합성을 깨지 않는다. 지갑 이중 생성 방지책의 핵심.

[6] VASP API 실패 → DB 미저장

npm run exercise:s27:6 · VASP 타임아웃 시 일관 상태 보장

```
$ npm run exercise:s27:6
```

실행 코드

```
const vasp = makeVaspClient({
  failGetWalletAddr: true // 강제 타임아웃
});
try {
  await svc.provision('K-20240008', 'EXTERNAL');
} catch (err) {
  console.log(err.message); // VASP network timeout
}
// saveCalls : 0 ← INSERT 없음
// DB rows    : 0 ← DB 오염 없음
```

실패 시퀀스

- ① getMapping() → null
- ② getWalletAddr() → throw Error (타임아웃)
- ③ saveMapping() → 미도달
→ DB rows = 0 ← 일관 상태 유지

왜 순서가 중요한가

잘못된 순서 (DB 먼저):

saveMapping() → VASP 실패 시 walletAddr = NULL인 행이 남음 →
역등성 체크에서 NULL 반환

올바른 순서 (VASP 먼저):

VASP 성공 확인 → saveMapping()
VASP 실패 시 DB 무결성 유지

패턴 이름: "외부 의존성 먼저, DB 저장 나중" — 롤백이 필요 없는 설계

[7] 감사 로그 + SHA-256 체크섬

npm run exercise:s27:7 · WALLET_PROVISIONED 이벤트 기록 · 먹등 시 미기록

```
$ npm run exercise:s27:7
```

감사 로그 구조

```
const raw = `${eventTime}${actor}${action}  
  ${resourceId}${JSON.stringify(afterState)}`;  
const checksum = createHash('sha256')  
  .update(raw, 'utf8')  
  .digest('hex');
```

예상 출력

```
[AUDIT] action      WALLET_PROVISIONED  
[AUDIT] resourceId  K-20240009  
[AUDIT] afterState  {"walletAddress":"0x...","vaspType":"EXTERNAL"}  
[AUDIT] checksum    a3f8... (SHA-256 64자)  
entries count : 1
```

재요청 후 : 1 ← 먹등, 감사 로그 미기록

핵심 포인트

SHA-256 목적: 감사 로그 위변조 탐지. eventTime + actor + action + resourceId + afterState 해싱 → 사후 수정 불가 증명.

먹등성 + 감사 로그: 재요청 시 getMapping()에서 기존 주소 반환 → auditLog.record() 미호출 → entries 증가 없음.

Optional Chaining: auditLog?.record() — auditLog 미주입 시 조용히 스킵. 테스트 환경 무로그 동작.

S27 · 지갑 프로비저닝 + 매핑 설계

Thank You

WalletProvisioningService · Repository 패턴 · DIP
먹등성 · VASP 분기 · SHA-256 감사 로그

M5 · S27

실습 완료

kyobo-digital-asset-platform

COINCRAFT
ACADEMY